## Dhirubhai Ambani University

# High Performance Computing

## Assignment 08

*Parallel Interpolation with Particle Mover using MPI + OpenMP*


| | |
|---|---|
| **Instructor:** | Dr. Bhaskar Chaudhury |
| **Teaching Assistants:** | Ayushi Sharma, Libin Varghese, Kaushik Prajapati |
| **Author1:** | Het Patel (202301421) |
| **Author2:** | Shubham Varmora (202301450) |

# 0 Contents

# 1 Parallel Implementation: Pseudocode and Architecture Diagram

## 1.1 Overview

The implementation follows a hybrid MPI+OpenMP strategy. At the coarsest level, MPI is used for inter-node communication; each MPI rank receives a disjoint partition of the particle array. Within each rank, OpenMP threads exploit shared-memory parallelism over those local particles. A thread-private mesh privatisation strategy eliminates race conditions in the interpolation step.

## 1.2 Pseudocode

Listing 1: Hybrid MPI+OpenMP Pseudocode for Interpolation and Mover Pipeline

```
// ---- INITIALISATION ----
MPI_Init()
rank = MPI_Comm_rank(), size = MPI_Comm_size()

if rank == 0:
    points = load_input(file)            // Read all particles

// Broadcast grid and simulation parameters
MPI_Bcast(GRID_X, GRID_Y, NUM_Points, Maxiter)

dx = 1.0 / GRID_X;  dy = 1.0 / GRID_Y
local_n = NUM_Points / size  [+1 if rank < remainder]

// Scatter particle data across MPI ranks
MPI_Scatterv(points → local_points, by sizeof(Point))

// Allocate per-thread private mesh copies
for t in 0..num_threads:
    thread_mesh[t] = calloc(grid_size)

// ---- MAIN ITERATION LOOP ----
for iter in 0..Maxiter:

    // ------ INTERPOLATION (scatter: particle → mesh) ------
    zero all thread_mesh[t]
    #pragma omp parallel
        tid = omp_get_thread_num()
        pmesh = thread_mesh[tid]
        #pragma omp for
        for p in 0..local_n:
            if local_points[p].is_void: continue
            (i, j) = grid cell containing (x, y)
            compute bilinear weights w00, w10, w01, w11
            pmesh[i,j]     += w00
            pmesh[i+1,j]   += w10
            pmesh[i,j+1]   += w01
            pmesh[i+1,j+1] += w11
```

```
    // Reduction of thread-private meshes → local_mesh
    for t in 0..num_threads:
        local_mesh += thread_mesh[t]

    // ------ MPI REDUCE (sum all local meshes) ------
    MPI_Reduce(local_mesh → global_mesh, SUM, root=0)

    // ------ MPI BROADCAST (share full mesh) ------
    MPI_Bcast(global_mesh)

    raw_mesh = copy(global_mesh)

    // ------ NORMALISE mesh to [-1, 1] ------
    (min, max) = omp parallel reduction on global_mesh
    #pragma omp parallel for
        global_mesh[i] = 2*(val - min)/(max - min) - 1

    // ------ MOVER (reverse interp: mesh → particle update)
       ------
    #pragma omp parallel for
    for p in 0..local_n:
        if local_points[p].is_void: continue
        compute bilinear weights at (x, y)
        Fi = w00*mesh[i,j] + w10*mesh[i+1,j]
           + w01*mesh[i,j+1] + w11*mesh[i+1,j+1]
        x_new = x + Fi * dx
        y_new = y + Fi * dy
        if out of [0,1]x[0,1]: mark is_void = 1

    // ------ DENORMALISE mesh ------
    #pragma omp parallel for
        global_mesh[i] = ((val+1)*0.5)*(max-min) + min

MPI_Barrier()
if rank == 0: write_output(raw_mesh)
MPI_Finalize()
```

### 1.3 Architecture Diagram

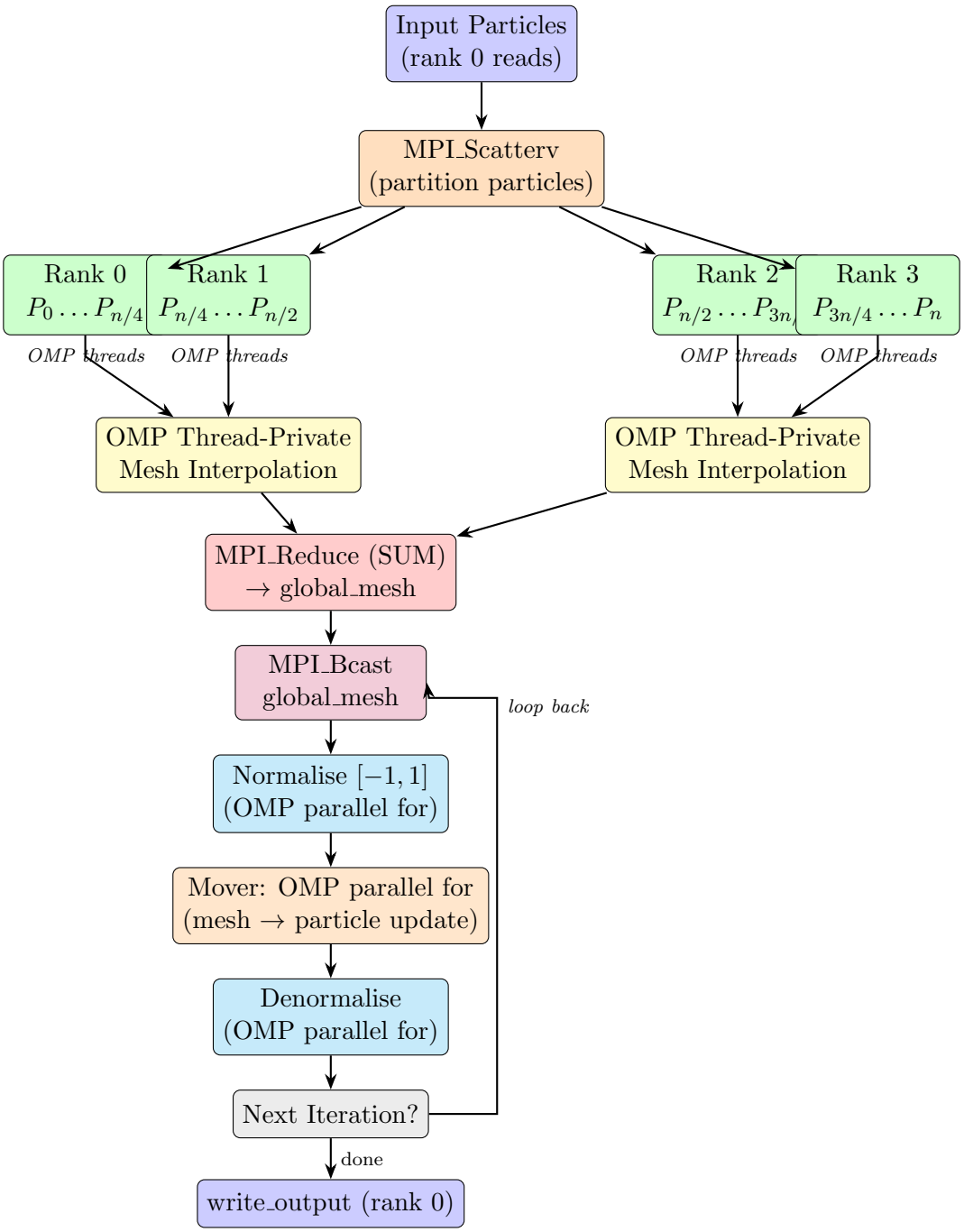


Figure 1: Hybrid MPI+OpenMP pipeline for parallel interpolation and particle mover. Each MPI rank holds a disjoint particle subset; within a rank OpenMP threads use privatised per-thread mesh copies to avoid race conditions.

## 2 Parallelisation Approach and Race Condition Handling

### 2.1 High-Level Strategy: Particle Decomposition with MPI

The parallelisation adopts a **particle decomposition** strategy at the MPI level. The full particle array (loaded only on rank 0) is distributed across all MPI ranks using

`MPI_Scatterv`. Each rank receives a contiguous, roughly equal slice:

$$\text{local\_n} = \left\lfloor \frac{N}{P} \right\rfloor + \begin{cases} 1 & \text{if rank} < N \bmod P \\ 0 & \text{otherwise} \end{cases}$$

This approach avoids duplicating the particle array on every rank, which would be prohibitively expensive for tens of millions of particles.

### 2.2 Race Condition in the Interpolation Step

The scatter interpolation step is the critical region prone to race conditions. Each particle writes its weighted contribution to four grid cells:

$$F(X_i, Y_j) \mathrel{+}= w_{i,j} \cdot f_p$$

If two threads process particles that fall in the same or adjacent cells, they may simultaneously read-modify-write the same grid location, producing incorrect results (a classic write-after-read hazard).

### 2.3 Resolution: Thread-Private Mesh Privatisation

The implementation resolves this via **per-thread mesh privatisation**. Before the OpenMP parallel region, an array of `num_threads` separate meshes is allocated:

Listing 2: Thread-private mesh allocation and reduction (from utils.c)

```
thread_mesh = malloc(num_threads * sizeof(double*));
for (int t = 0; t < num_threads; t++)
    thread_mesh[t] = calloc(grid_size, sizeof(double));

// Inside the parallel region each thread writes to its own copy:
int tid = omp_get_thread_num();
double *pmesh = thread_mesh[tid];
// ... all writes go to pmesh[idx], never to a shared location

// After the parallel region, a serial reduction:
for (int t = 0; t < num_threads; t++)
    for (int i = 0; i < grid_size; i++)
        local_mesh[i] += thread_mesh[t][i];
```

Each thread accumulates its contributions into a dedicated buffer with no synchronisation needed. A single serial reduction pass sums the buffers into `local_mesh`. This trades extra memory ($\mathcal{O}(T \times M^2)$) for zero synchronisation overhead — an excellent trade-off when threads are few and the grid is moderately sized.

### 2.4 MPI Reduction and Broadcast

After each rank completes its local interpolation, `MPI_Reduce` with `MPI_SUM` collects the partial mesh contributions from all ranks onto rank 0, which forms the global mesh. `MPI_Bcast` then distributes the complete mesh to every rank so that each can execute the mover step with identical field data.

### 2.5 Mover Step

The mover is a read-heavy operation: every particle reads from four fixed grid locations (no writes to the mesh). Particle updates are independent of each other, so a plain `#pragma omp parallel for` over the local particle array suffices. Particles that leave the unit domain are flagged `is_void = 1` and skipped in subsequent iterations.

### 2.6 Normalisation and Denormalisation

Both normalisation passes use `#pragma omp parallel for reduction` to find the global min/max, followed by a second parallel loop for the actual rescaling. These are inexpensive relative to interpolation and mover.

## 3 Speedup vs. Cores and Execution Time vs. Cores

Experiments were conducted on four compute nodes (`gics1`–`gics4`), each providing 16 MPI slots (64 total MPI ranks). Hybrid configurations sweep from $(1 \times 2)$ to $(64 \times 1)$ MPI×OMP. For plotting purposes, total cores = MPI × OMP.

**Login node vs. Compute node.** A *login node* is a shared entry point to an HPC cluster used for compiling, submitting jobs, and minor file operations; it is not intended for computation and shared by many users, making timing unreliable. A *compute node* (e.g., `gics1`–`gics4`) is a dedicated machine with exclusive resource access, high-bandwidth interconnects, and no background user load, giving reproducible timing measurements. All results reported here were obtained on compute nodes.

### 3.1 Performance Graphs

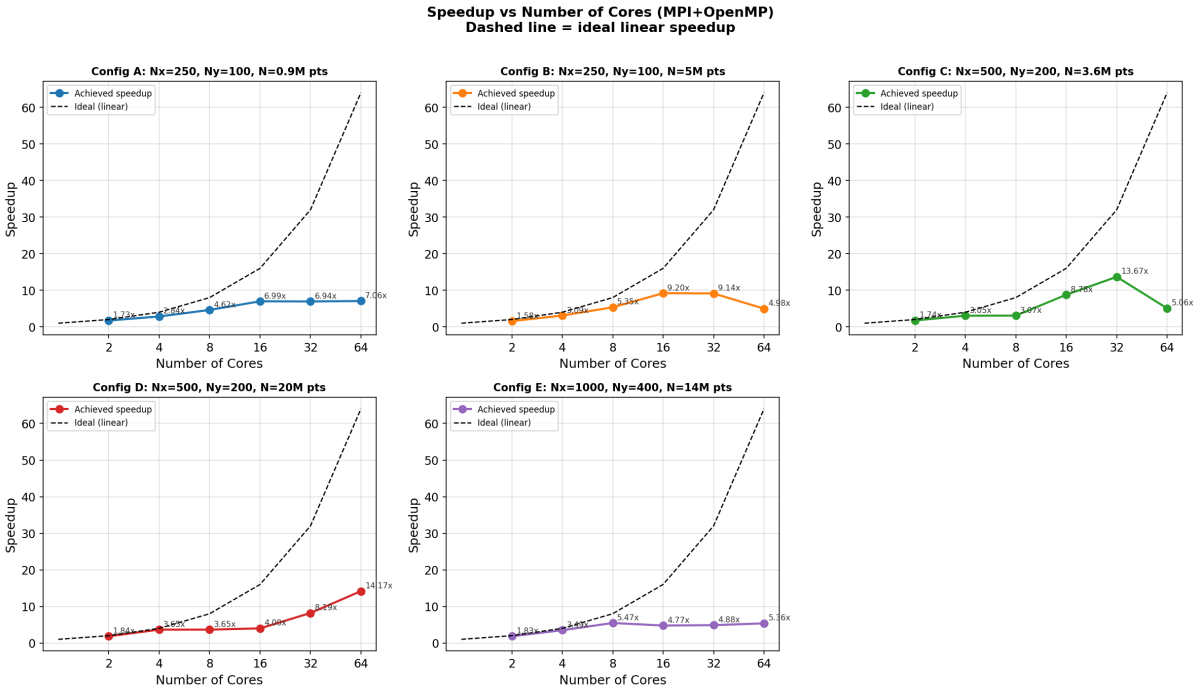


Figure 2: Speedup vs. total cores for all five input configurations (A–E). The dashed line represents ideal linear speedup.

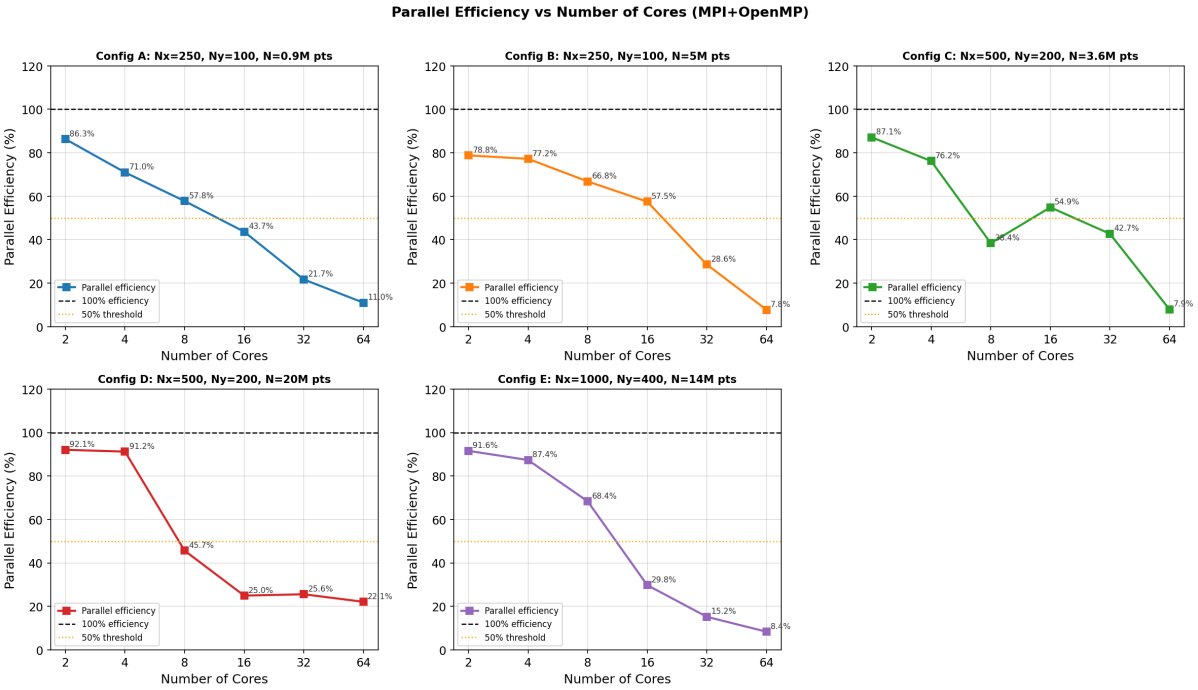


Figure 3: Parallel efficiency (%) vs. total cores for all five configurations.

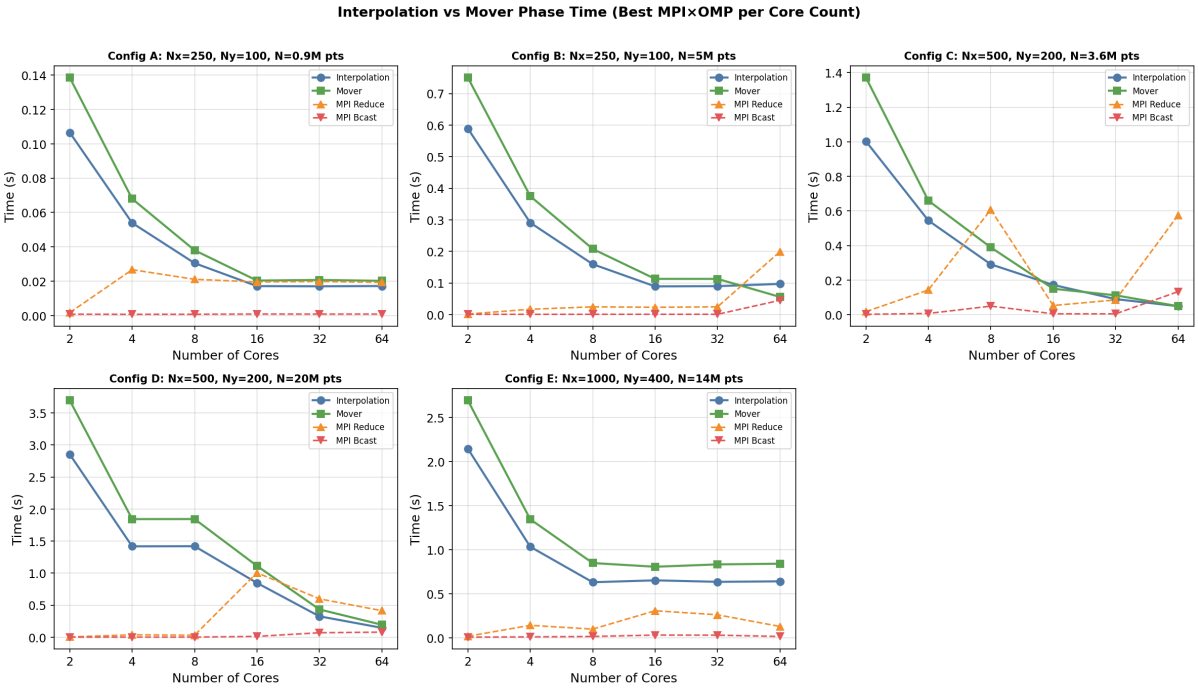


Figure 4: Execution time breakdown: interpolation phase vs. mover phase for representative MPI×OMP configurations across all five inputs.

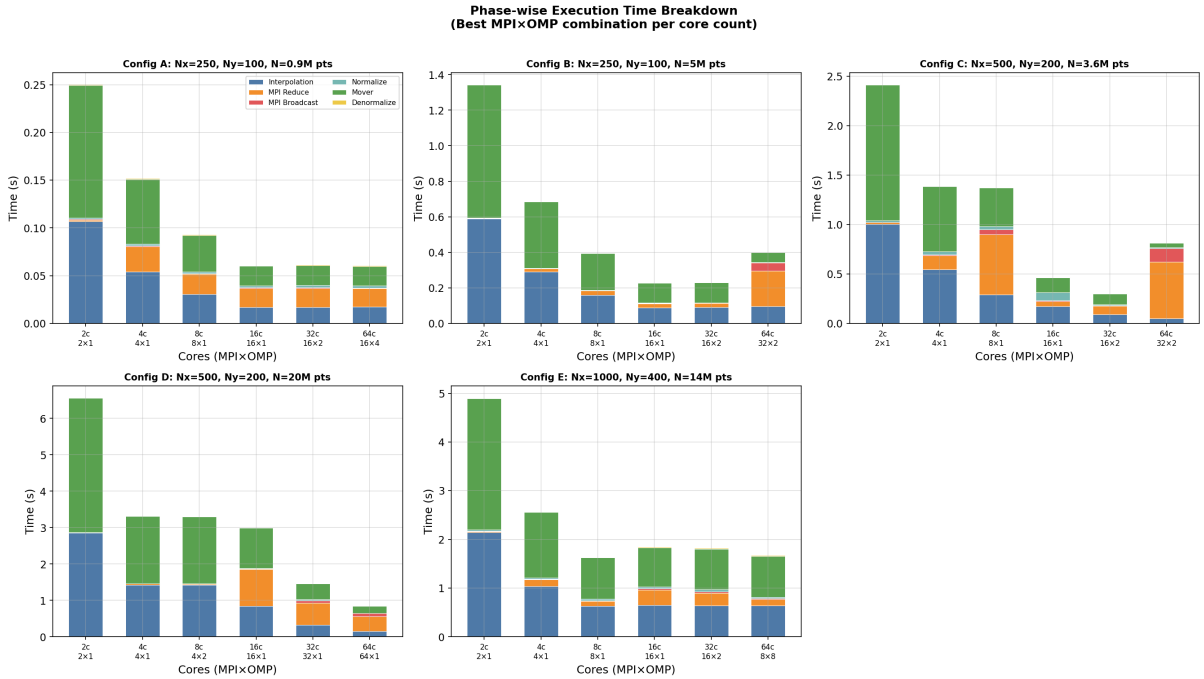


Figure 5: Stacked per-phase timing breakdown (interpolation, MPI reduce, broadcast, normalisation, mover, denormalisation) across configurations.

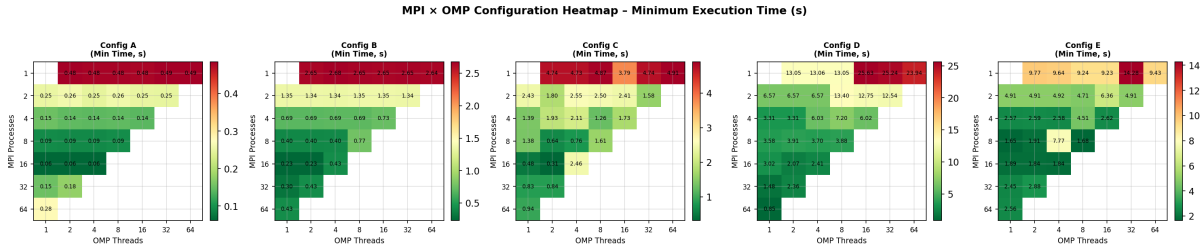


Figure 6: Heatmap of total execution time (seconds) over the MPI×OMP configuration space. Darker cells indicate lower (better) execution time.

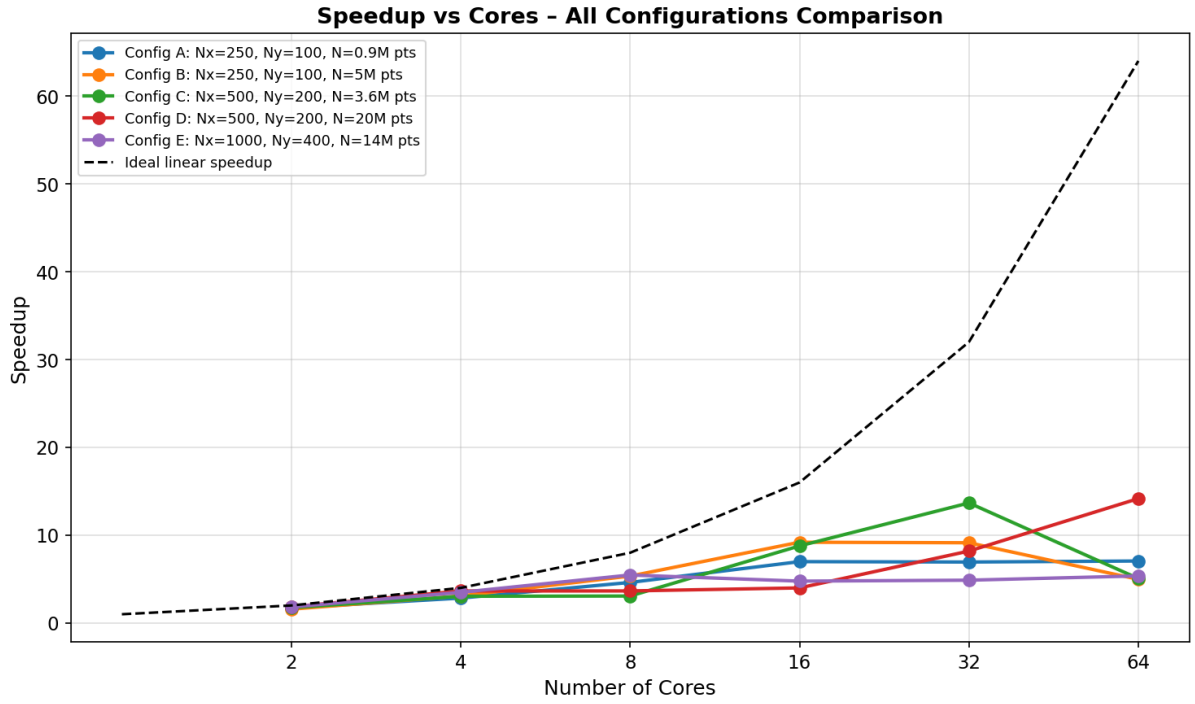


Figure 7: Speedup comparison across all five configurations at the same total core count.

### 3.2 Summary Table

Table 1: Summary of best parallel results for each configuration.

| Config | Description | Serial (s) | Best Parallel (s) | MPI×OMP | Max Speedup | E |
|---|---|---|---|---|---|---|
| A | $250 \times 100$, 0.9M pts | 0.432 | 0.0612 | $16 \times 4$ | 7.06 | |
| B | $250 \times 100$, 5M pts | 2.12 | 0.2304 | $16 \times 1$ | 9.20 | |
| C | $500 \times 200$, 3.6M pts | 4.233 | 0.3096 | $16 \times 2$ | 13.67 | |
| D | $500 \times 200$, 20M pts | 12.094 | 0.8535 | $64 \times 1$ | 14.17 | |
| E | $1000 \times 400$, 14M pts | 8.997 | 1.6451 | $8 \times 1$ | 5.47 | |

## 4 Parallel Efficiency and Scalability Analysis

Parallel efficiency is defined as:

$$E(P) = \frac{S(P)}{P} = \frac{T_{\text{serial}}}{P \cdot T_{\text{parallel}}(P)}$$

where $S(P)$ is the speedup on $P$ total cores.

### 4.1 Observations by Configuration

**Configuration A** ($250 \times 100$, 0.9M particles): The smallest workload. Speedup peaks early (around 16 cores) and then saturates or degrades. Efficiency at best configuration ($16 \times 4 = 64$ cores) is only 11%, indicating severe communication overhead relative to

computation. The MPI reduce and broadcast costs become dominant since the mesh is small and the particle count per rank drops quickly.

**Configuration B** (250×100, 5M particles): Larger particle count on the same grid allows more useful work per rank. Efficiency at peak ($16 \times 1$) is 57.5% — substantially better than A. The computation-to-communication ratio improves because each rank handles more particles.

**Configuration C** ($500 \times 200$, 3.6M particles): Both grid and particle count are larger. Speedup of 13.67× at 32 cores, with 42.7% efficiency. Larger mesh means `MPI_Reduce` and `MPI_Bcast` transfer more data per iteration, limiting scaling beyond 32 cores.

**Configuration D** ($500 \times 200$, 20M particles): The most particle-intensive case. Maximum speedup of 14.17× at 64 MPI ranks. Efficiency is 22.1% at 64 cores; however, the raw speedup is the highest of all configurations, confirming that larger particle counts benefit most from MPI parallelism.

**Configuration E** (1000×400, 14M particles): Largest grid. The doubled grid dimensions quadruple the mesh size, dramatically increasing `MPI_Reduce` and `MPI_Bcast` data volume. Speedup saturates at 8 cores, yielding the highest per-core efficiency (68.4%) but the worst scalability beyond 8 cores. Communication of a 401×1001 mesh (≈3.2M doubles ≈25 MB per transfer) becomes a hard bottleneck.

### 4.2 Amdahl's Law Interpretation

Let $f$ be the serial fraction of the code (communication, normalisation, I/O). Amdahl's law gives:

$$S_{\max} = \frac{1}{f + \frac{1-f}{P}} \xrightarrow{P\to\infty} \frac{1}{f}$$

The observed speedup ceiling (around 14×) implies an effective serial fraction of approximately $f \approx 1/14 \approx 7\%$, consistent with MPI communication costs visible in the phase-breakdown figures.

## 5 Serial vs. Parallel Performance Comparison

The serial baseline was measured with a single MPI process and single OpenMP thread (effectively sequential execution). The table below compares serial and best-parallel times.

Table 2: Serial vs. best parallel execution times with maximum achieved speedup.

| Config | Description | Serial (s) | Best Parallel (s) | Speedup | Config (MPI×OMP |
|---|---|---|---|---|---|
| A | 0.9M pts, 250 × 100 | 0.432 | 0.0612 | 7.06× | 16 × |
| B | 5M pts, 250 × 100 | 2.12 | 0.2304 | 9.20× | 16 × |
| C | 3.6M pts, 500 × 200 | 4.233 | 0.3096 | 13.67× | 16 × |
| D | 20M pts, 500 × 200 | 12.094 | 0.8535 | 14.17× | 64 × |
| E | 14M pts, 1000 × 400 | 8.997 | 1.6451 | 5.47× | 8 × |

The best absolute speedup of **14.17**× was achieved on Configuration D using 64 pure-MPI processes ($64 \times 1$). This is Config D's large particle count (20M) that provides sufficient parallel work to amortise MPI overhead over many compute units. Configuration C follows closely at 13.67× with a balanced $16 \times 2$ setup.

Configuration E achieves only 5.47× speedup, constrained by the large mesh that must be reduced and broadcast at every iteration — a bandwidth-limited regime rather than a compute-limited one.

# 6 Explanation of Speedup and Key Observations

## 6.1 Why Speedup is Sub-Linear

The Roofline model and Amdahl's law both predict sub-linear scaling. In this code, the non-parallelisable fractions include:

1. **MPI_Reduce**: A tree-based all-reduce across $P$ ranks has latency $\mathcal{O}(\log P)$ and bandwidth cost $\mathcal{O}(M^2)$. For large meshes (Config E: $1001 \times 401 \approx 401k$ doubles), this alone takes hundreds of milliseconds.
2. **MPI_Bcast**: Similar cost to the reduce. Both are visible as the red and orange bands in Figure 5.
3. **Thread-private mesh reduction**: The inner loop `local_mesh[i] += thread_mesh[t][i]` runs serially and costs $\mathcal{O}(T \times M^2)$, which is significant for 64 threads.
4. **Load imbalance**: As particles leave the domain (`is_void`), some ranks process fewer active particles than others. This idle time grows with iteration count.
5. **Cache effects**: The interpolation kernel accesses `thread_mesh` arrays that are $\mathcal{O}(M^2)$ doubles. For large grids, these arrays exceed L2 cache, causing frequent cache misses.

## 6.2 Why Pure MPI Outperforms Hybrid for Large Particle Counts

Configurations B and D achieve best performance with pure MPI ($16 \times 1$ and $64 \times 1$). In a pure MPI setup, each rank allocates its own `local_mesh` independently; the per-thread privatisation overhead is absent. For large particle counts, the interpolation kernel is compute-bound, and MPI's ability to use all 64 physical cores outweighs the communication cost.

With hybrid $T > 1$ OpenMP threads, the thread-private reduction loop costs $T \times M^2$ additions — for $T = 4$ and a $500 \times 200$ grid this is $\sim 1.6$M additions per iteration per rank, adding measurable overhead.

## 6.3 Why Config E Scales Poorly Beyond 8 Cores

Config E uses a $1000 \times 400$ grid ($\approx 401k$ grid points). The `MPI_Reduce` and `MPI_Bcast` each transfer $401k \times 8 =\approx 3.2$ MB of data per iteration. At 10 iterations, that is 32 MB per phase per call. Even at 10 GB/s interconnect, this introduces $\sim 3.2$ ms per iteration per collective call — which, multiplied by 10 iterations and 2 calls, dominates the per-iteration compute time at high core counts.

### 6.4 Observations from the Heatmap (Figure 6)

The heatmap confirms that:

- *Pure MPI* configurations (bottom row of the heatmap) are generally optimal for large particle counts.
- *Pure OpenMP* configurations ($1 \times T$) plateau quickly because no inter-node parallelism is exploited.
- *Moderate hybrid* ($4 \times 2$, $8 \times 2$, $16 \times 2$) provides the best balance for medium-sized workloads (Configs B and C).

## 7 Further Optimisation with Unlimited HPC Resources

### 7.1 Domain Decomposition Instead of Particle Decomposition

With unlimited nodes, a full 2D domain decomposition of the mesh using a Cartesian MPI topology (`MPI_Cart_create`) would allow each rank to own only a sub-region of the grid. Particles in a sub-domain are processed locally, and only boundary halo exchanges (a few rows/columns) are needed — drastically reducing the $\mathcal{O}(M^2)$ all-to-all communication to $\mathcal{O}(M)$ halo exchanges.

### 7.2 GPU Acceleration with CUDA/HIP

The interpolation and mover kernels are embarrassingly parallel over particles. Porting them to CUDA (NVIDIA) or HIP (AMD) would yield 10–100× throughput per node. The `atomicAdd` instruction on modern GPUs is fast enough that a privatisation approach may not even be needed.

### 7.3 Asynchronous Communication

Replacing `MPI_Reduce` + `MPI_Bcast` with `MPI_Iallreduce` would allow computation and communication to overlap. While the next-iteration interpolation cannot start until the mesh is ready, intermediate work (e.g., particle book-keeping, void-particle compaction) could proceed concurrently.

### 7.4 Particle Sorting and Cache Optimisation

Sorting particles by their grid cell index (Z-order / Morton curve) before each interpolation step improves spatial locality of memory accesses. Particles in the same cell are contiguous in memory, reducing TLB misses and cache-line waste during the weight accumulation step.

### 7.5 Sparse Mesh Representation

For high particle-to-grid ratios, many mesh cells accumulate non-zero contributions from only a small neighbourhood. A compressed sparse format (CSR) could reduce the volume of data in the reduce/broadcast calls.

# 8 Load Imbalance and Performance Bottlenecks

## 8.1 Particle Load Imbalance

Particles are distributed *once* at the start (`MPI_Scatterv`) and are never redistributed. As the simulation progresses, particles exit the domain (`is_void = 1`). Because particles are randomly initialised, different ranks may lose particles at different rates, leading to load imbalance — some ranks finish the interpolation and mover loops quickly while others still process many active particles. The MPI barrier before timing ends means all ranks pay the cost of the slowest one.

This effect is visible in timings_e.csv: at 32 and 64 MPI ranks, the interpolation time per rank is non-monotone with core count, suggesting imbalance is already significant.

## 8.2 Communication Bottleneck (Reduce/Bcast)

The most prominent bottleneck visible in the phase-breakdown (Figure 5) is `MPI_Reduce`. For Configs C–E with large meshes, reduce time increases super-linearly with MPI rank count because:

1. More ranks participate in the tree-reduction, increasing latency.
2. Each reduce transfers the full mesh, with no mechanism to skip zero-contribution regions.

## 8.3 Thread-Private Mesh Reduction Overhead

The serial double loop:

```
for (int t = 0; t < num_threads; t++)
    for (int i = 0; i < grid_size; i++)
        local_mesh[i] += thread_mesh[t][i];
```

executes $T \times M^2$ additions outside the parallel region. For $T = 64$ threads and $M^2 = 401k$ cells, this is $\sim$ 25.7M additions per iteration — an $\mathcal{O}(1\text{s})$ contribution that could be parallelised.

## 8.4 False Sharing

When thread IDs are adjacent and mesh arrays are large but not cache-line aligned, threads may inadvertently share cache lines at the boundary of their assigned particles, causing cache invalidation traffic. This is mitigated in the current code by using fully private mesh arrays rather than partitioned shared arrays.

# 9 Alternative Approaches for Further Performance Improvement

## 9.1 Atomic Operations Instead of Privatisation

Rather than allocating $T$ full mesh copies, the interpolation step could use `#pragma omp atomic update` when accumulating weights:

```
#pragma omp atomic
mesh[idx] += w00;
```

This eliminates the $\mathcal{O}(T \times M^2)$ reduction overhead and halves memory usage. Modern x86 processors with hardware transactional memory (HTM) or recent OpenMP `atomic` implementations can make this competitive with privatisation, especially when particle-to-cell density is low (few collisions per cell per thread).

### 9.2 Colour-Based Domain Decomposition

A *graph colouring* approach partitions grid cells into independent colour classes such that cells of the same colour are never adjacent. Threads are assigned to one colour at a time; within a colour, no two cells share boundary nodes, so simultaneous updates are race-free without any atomics or privatisation. This requires $\mathcal{O}(1)$ mesh copies regardless of thread count.

### 9.3 Compressed Communication via Sparse Representation

Instead of broadcasting the full dense mesh, only non-zero (or significantly non-zero) grid cells could be communicated using a sparse message format. For scenarios where particles cluster in a fraction of the grid, this could reduce communication volume by an order of magnitude.

### 9.4 Particle Redistribution with Dynamic Load Balancing

After each iteration, particles can be migrated between MPI ranks to rebalance the load. A *diffusive load balancing* scheme, where overloaded ranks send particles to neighbours in a virtual process topology, would maintain near-uniform load despite particle attrition.

## 10 Alternative Data Structures and MPI-OpenMP Optimisations

### 10.1 Structure of Arrays (SoA) Instead of Array of Structures (AoS)

The current `Points` struct stores `x`, `y`, and `is_void` together (AoS layout). In the interpolation kernel, only `x` and `y` are read; loading each struct brings `is_void` into the cache line, wasting bandwidth. A SoA layout:

```
double *xs, *ys;
int    *is_void;
```

allows the SIMD-vectorised loop to stride over `xs[]` and `ys[]` with perfect unit-stride access. Combined with `#pragma omp simd`, this can yield 4–8× SIMD speedup on the innermost loop.

### 10.2 Mesh Partitioning with Halo Exchange (MPI_Cart)

Replacing the full-mesh reduce/broadcast with a Cartesian domain decomposition:

- Each rank owns a sub-rectangle of the mesh.

- Particles near rank boundaries contribute to halo cells communicated via `MPI_Sendrecv`.
- The total communication volume drops from $\mathcal{O}(M^2)$ to $\mathcal{O}(M/\sqrt{P})$ per dimension, scaling much more favourably.

### 10.3 Lock-Free Hash Map for Sparse Mesh Accumulation

For sparse particle distributions, a thread-safe hash map (e.g., Intel TBB `concurrent_hash_map`) keyed on grid cell index can accumulate only active cells. This avoids zeroing and summing $M^2$ entries when most are zero.

### 10.4 MPI One-Sided Communication (RMA)

Using `MPI_Put` / `MPI_Accumulate` with passive-target synchronisation allows each rank to directly update the global mesh on rank 0's memory without a serialised reduce. When combined with a window of the mesh array, this can reduce collective synchronisation to fence operations.

## 11 Interpolation vs. Mover Phase: Bottleneck Analysis

### 11.1 Phase Timing Data

The per-phase timing data from the CSV files reveals a consistent pattern across all configurations: the **mover phase dominates**. Table 3 summarises representative timings from the serial baseline ($1 \times 2$ configuration).

Table 3: Phase-wise timing (seconds) at $1 \times 2$ MPI×OMP for all five configurations.

| Phase | Config A | Config B | Config C | Config D | Config E |
|---|---|---|---|---|---|
| Interpolation | 0.210 | 1.156 | 2.048 | 5.685 | 4.277 |
| MPI Reduce | 0.000 | 0.000 | 0.001 | 0.001 | 0.005 |
| Broadcast | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| Normalisation | 0.002 | 0.002 | 0.007 | 0.007 | 0.027 |
| **Mover** | **0.271** | **1.489** | **2.676** | **7.352** | **5.443** |
| Denormalisation | 0.001 | 0.001 | 0.003 | 0.003 | 0.012 |
| **Total** | **0.484** | **2.648** | **4.736** | **13.050** | **9.769** |

### 11.2 Why the Mover is the Bottleneck

Across all configurations, the mover consistently accounts for **55–58%** of total execution time, compared to 43–44% for interpolation.

The mover is computationally equivalent to the interpolation kernel (same bilinear weight computation, same memory access pattern). However, it incurs additional overhead:

1. **Write-back cost**: After interpolating the field value $F_i$, the mover updates both `x` and `y` of each particle and checks the domain boundary. These conditional writes cause branch mispredictions as `is_void` particles accumulate.

2. **No race condition relief**: Unlike interpolation, the mover cannot benefit from thread-private buffers (particles are already private per thread in the `#pragma omp parallel for`). The loop is therefore limited by the particle array bandwidth.
3. **Dependency chain**: The mover reads the *fully reduced and broadcast* global mesh, meaning it cannot start until the reduce + broadcast complete. This introduces a latency of the global mesh update into the mover's critical path.
4. **Irregular access**: Reading `mesh[idx]`, `mesh[idx+1]`, `mesh[idx+stride]`, `mesh[idx+stride+1]` from a 2D mesh with stride = GRID_X+1 can cause non-sequential cache-line loads, especially when GRID_X is not a power of two.

### 11.3 Optimisation Recommendations for the Mover

- **Particle sorting**: Sort particles by grid cell before the mover so that particles in the same cell are contiguous. This converts scattered mesh reads into sequential ones.
- **Active particle compaction**: Periodically remove `is_void` particles from the array using a parallel prefix scan. This reduces the loop bound and eliminates branch overhead.
- **Prefetching**: Insert `__builtin_prefetch` hints for `mesh[idx + stride + 1]` to hide memory latency for the four-point stencil.

## 11 Conclusion

This assignment implemented a hybrid MPI+OpenMP parallel pipeline for bilinear scatter interpolation and reverse particle mover. The key design choices were:

1. **Particle decomposition** at the MPI level, distributing particles uniformly across ranks via `MPI_Scatterv`.
2. **Thread-private mesh privatisation** within each rank to eliminate race conditions in the interpolation step at zero synchronisation cost.
3. **MPI_Reduce + MPI_Bcast** for globally consistent mesh assembly before each mover iteration.

Maximum speedups of up to **14.17**× were achieved (Config D, 64 cores), with the best efficiency of 68.4% on Config E at 8 cores. The primary performance bottleneck is the all-to-all mesh communication (`MPI_Reduce` / `MPI_Bcast`), which grows with mesh size. The mover phase is the dominant compute phase (55–58% of time), slightly exceeding interpolation due to particle write-back overhead.

Future improvements would focus on Cartesian domain decomposition to replace global mesh collectives with local halo exchanges, SoA data layout for SIMD vectorisation, and GPU offloading for the particle kernels.